Walchand College of Engineering, Sangli

Dept. of Computer Science and Engineering, TY (A)

MINI PROJECT-I (7CS345) — FINAL EVALUATION,
2026-27

PROMPTOPT - TOKEN-EFFICIENT PROMPT OPTIMIZATION FOR LLMS

Team Members

Piush U. Gogi - 255200005

Nirzara A. Manade - 255200006

Siddhi S. Bolaikar - 255200008

Under the Guidance of -

Prof. A.S. Pawar

AGENDA

1. The Problem
2. What Exists & the Gap
3. Our Goal
4. Our Data
5. One Example, Start to Finish
6. Results
7. Testing & the Working App
8. Challenges & Next Steps

THE PROBLEM

People type short, vague prompts. The AI has to guess, so it writes long answers nobody asked for, and every extra word costs tokens, time and money.

A REAL EXAMPLE

The user types: `“hey can you just tell me if this review is good or bad”`
+ a short phone review

The AI answers with a whole page: a heading, positive points, negative points and a closing paragraph.

166 tokens in the answer alone

The user only wanted one word: good or bad.

OUR SURVEY OF STUDENTS AND DEVELOPERS

72% run out of tokens very quickly

58% need several prompts to get one answer

50% often or very often ask the AI again

GAP ANALYSIS

Feature	PO.tools	Aiven	BenchLM	SiteGPT	PromptOpt
Rewrite prompts	✓	✓	✓	✓	✓
Goal/Framework customization	Limited	✓	Limited	✓	✓
Preserve constraints	✗	-	✓	-	✓
LLM output evaluation	✗	✗	✗	✗	✓
Token efficiency report	-	✓	-	-	✓
Multi-stage pipeline	✗	✗	✗	✗	✓
Comprehensive reporting	✗	✗	Partial	✗	✓
Access	Free	Free	Limited	Free	**Self-hosted**

Key Gaps Identified →

- **No automated Evaluation of optimized prompts to actual prompts**
- **Limited or unclear handling of constraints and requirements**
- **No systematic measurement of token efficiency with quality trade-offs**
- **Absence of multi stage optimization pipeline combining rule based and Lora methods**

OUR GOAL

1. **To find what a prompt is missing**

   answer format, length, language, or which text “this” means

2. **To fix it with simple rules**

   and show every change, before and after

3. **To use a small AI model only when the rules are stuck**

   trained by us, checked before it is used

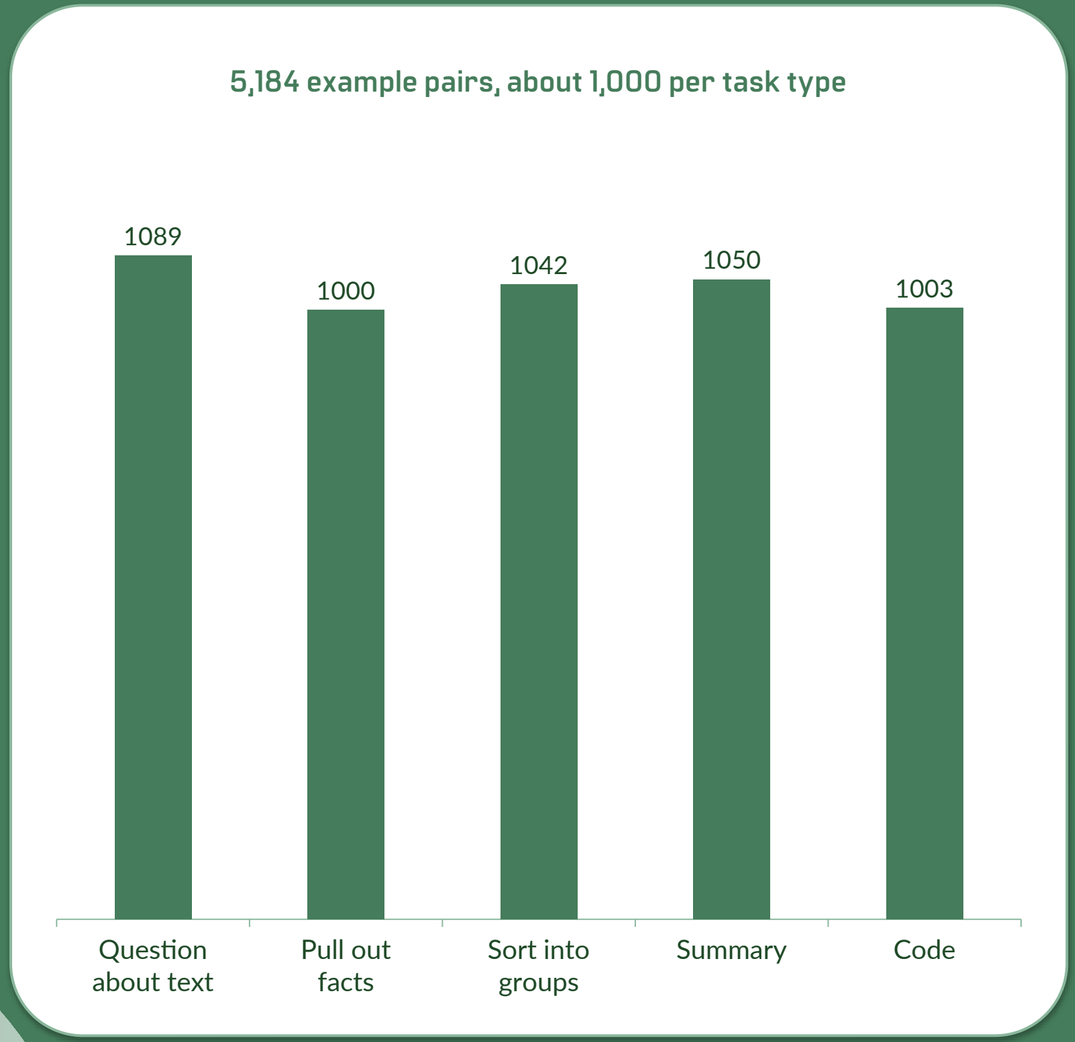
4. **To write the prompt the way each AI likes it**

   GPT, Gemini or Claude: same content, their layout

5. **To prove it with numbers**

   fewer tokens, same or better answers, on prompts never seen before

OUR DATA



1 **Real requests**

from two public datasets: Dolly (text tasks) and CodeAlpaca (code)

2 **Vague + clear version**

an AI wrote how a rushed user would type it, and a clear version

3 **Checked by computer**

drift, missing data, too long; dropped data put back

4 **Checked by us**

330 pairs read by the three of us and our guide: 295 kept

5 **Split once**

4,509 to learn from · 149 to tune · 482 to test, used only once

ONE EXAMPLE, START TO FINISH

1. The user types

“hey can you just tell me if this review is good or bad”
+ pasted review: “The battery lasts two days and the camera is great, but the phone heats up while gaming.”

prompt + review

2. Stage A: understand

Task type? summarization 0.52,
classification 0.35: below 0.6, not sure
Filler: “hey”, “can you”, “just”
No answer format stated
Text is attached

what is missing

3. Stage B: fix with rules

B07 tidy + B01 remove filler:
“Tell me if this review is good or bad.”
Category not sure: no rule guesses a format. Task category unresolved, so the prompt goes to Stage C

unresolved: task category

4. Stage C: small trained model

Asked for: format, constraints, category
Answer passed every check:
format: “Provide only the label.”
category: “classification”
The user is asked to confirm it

final prompt

5. Written for the target LLM

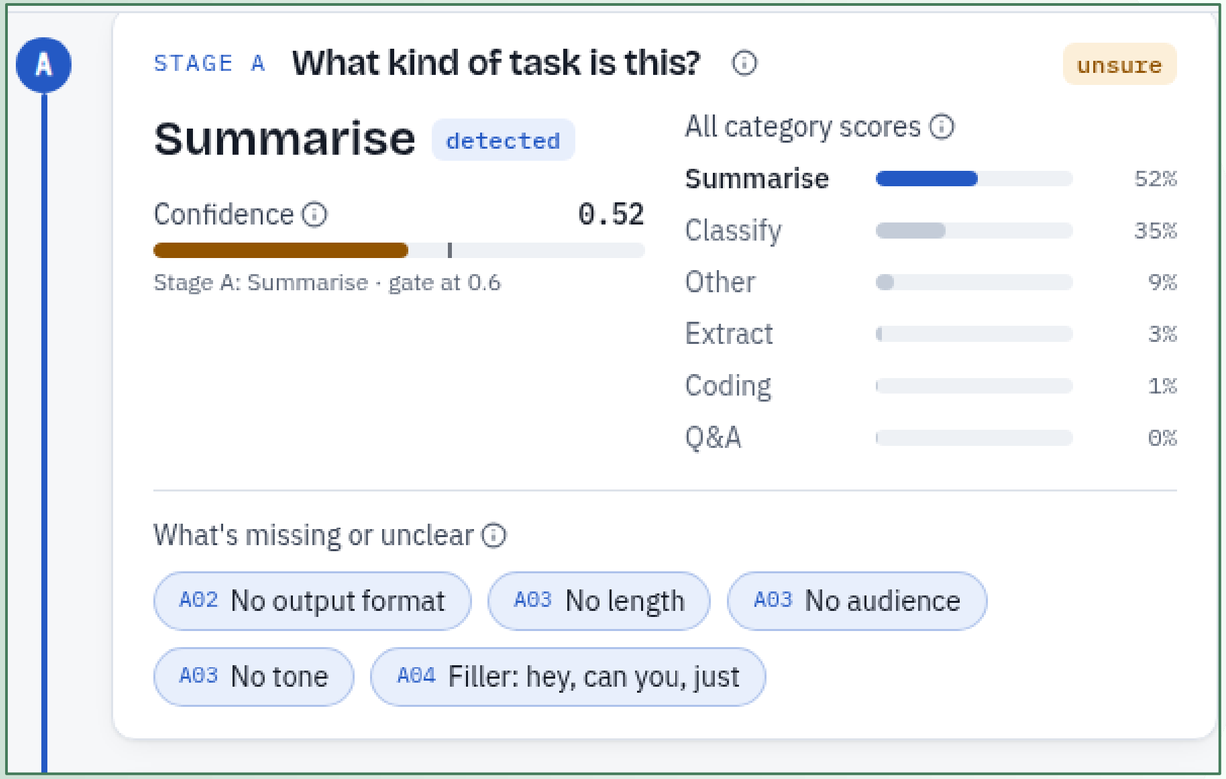
Task
Tell me if this review is good or bad.
Context (the review)
Output format
Provide only the label.

GPT rendering

6. Compare on a real model

OpenAI GPT model (gpt-oss-120b)
Vague prompt: 166-token explanation
Optimized prompt: “good” (41 tokens)
Total tokens: 270 vs 162 (−40%)
Time: 353 ms vs 97 ms

STEP A: UNDERSTAND THE PROMPT



What kind of task is it?

Summary 52%, sorting into a label 35%. Below 60% means “not sure”, so nothing will be guessed.

What is missing?

No answer format. Filler words: “hey”, “can you”, “just”.

How does it decide?

It compares the prompt with 9,000+ example prompts from our data and checks it with word rules.

STEP B: FIX WITH SIMPLE RULES

B

STAGE B **Rules that fired** 2 changes

B07 Tidy the structure

Material pasted inside the prompt goes into its own block; the task gets a capital and a full stop.

~~hey~~Hey can you just tell me if this review is good or ~~bad~~bad.

B01 Remove filler

Words like “hey can you just” cost tokens and say nothing to the model.

~~Hey can you just tell~~Tell me if this review is good or bad.

Left unresolved: `task category` `output format`

BEFORE

```
hey can you just tell me if this review is good or bad
```

AFTER

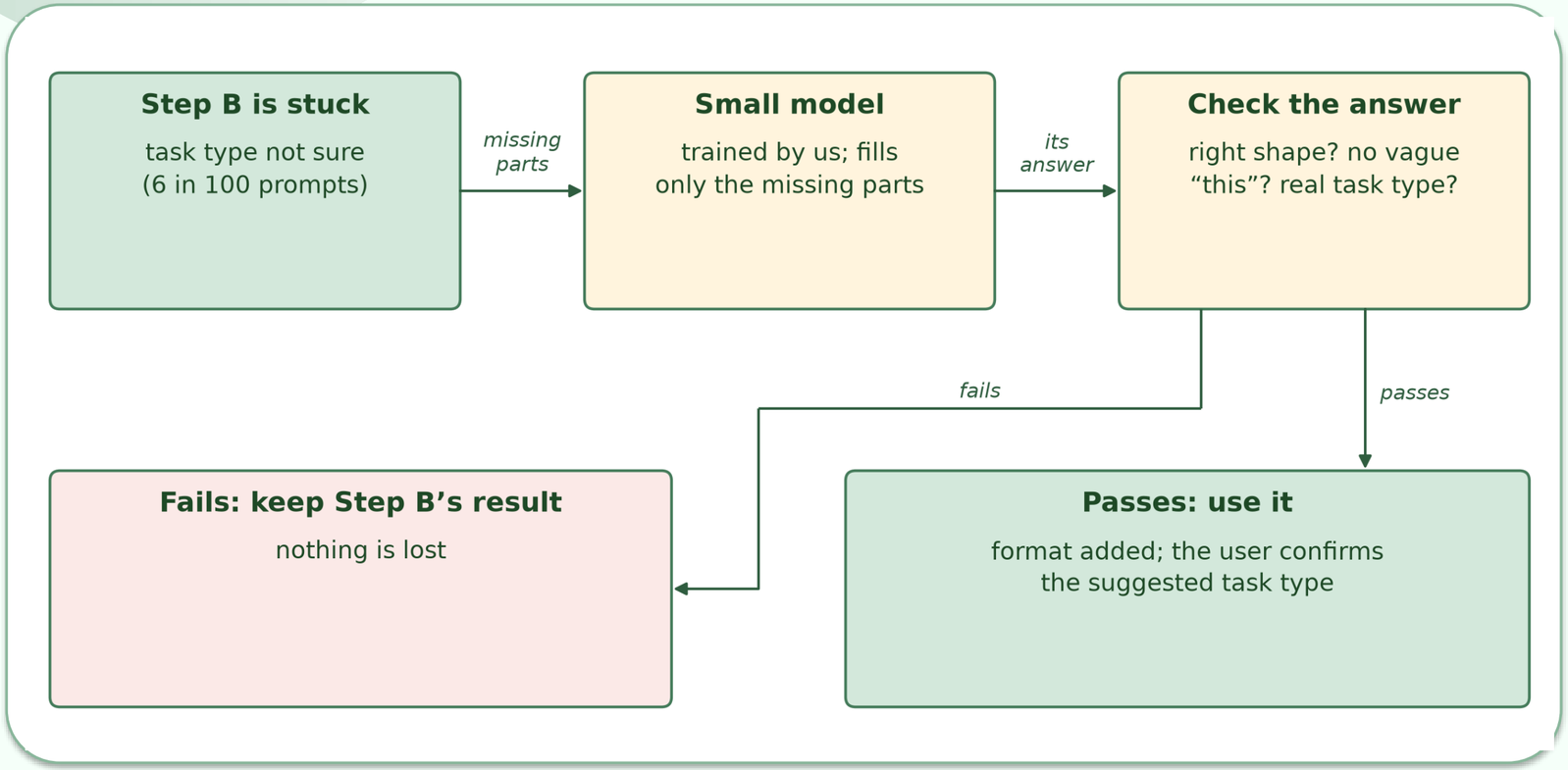
```
Tell me if this review is good or bad.
```

15 SMALL RULES, EACH ONE TESTED

remove filler	remove repeats	move pasted data
add answer format	add length	add language
list the labels	answer from the text	attachments

Not sure of the task? Then no rule adds a format. The question goes to Step C.

STEP C: A SMALL MODEL, ONLY WHEN STUCK



For our example it added **"Provide only the label."** and suggested **"sort into a label"**. The app asks the user to confirm.

6 in 100

prompts need Step C; the rules handle the rest

ANSWERS THAT PASS THE CHECK

5 / 100 before training

98 / 100 after our training

51 min

to train on a 4 GB laptop GPU

STEP D: WRITTEN FOR YOUR AI

Claude

```
<context>
<document>
The battery lasts two
days and the camera is
great, but the phone
heats up while gaming.
</document>
</context>
<task>
Tell me if this review
is good or bad.
</task>
<output_format>
Provide only the label.
</output_format>
```

GPT

```
### Task
Tell me if this review
is good or bad.

### Context
Document:
```
The battery lasts two
days and the camera ...
```

### Output format
Provide only the label.
```

Gemini

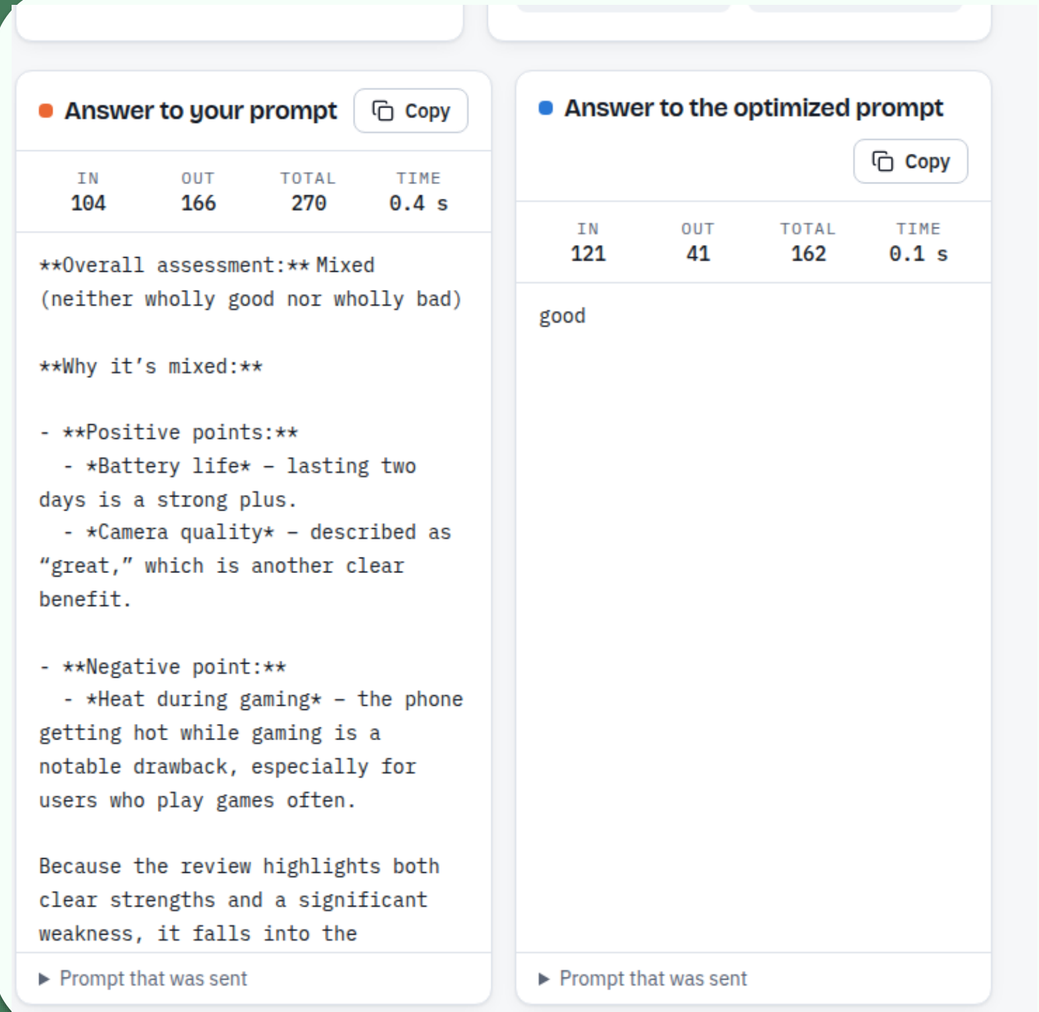
```
Task: Tell me if this
review is good or bad.

Output format: Provide
only the label.

Context:
Document:
```
The battery lasts two
days and the camera ...
```
```

Same content for every AI, in the layout its makers recommend. Our prompt for GPT: 50 tokens, counted exactly.

THE RESULT FOR OUR EXAMPLE



Total tokens

270
vague prompt

162
optimized

Tokens in the answer

166
vague prompt

41
optimized

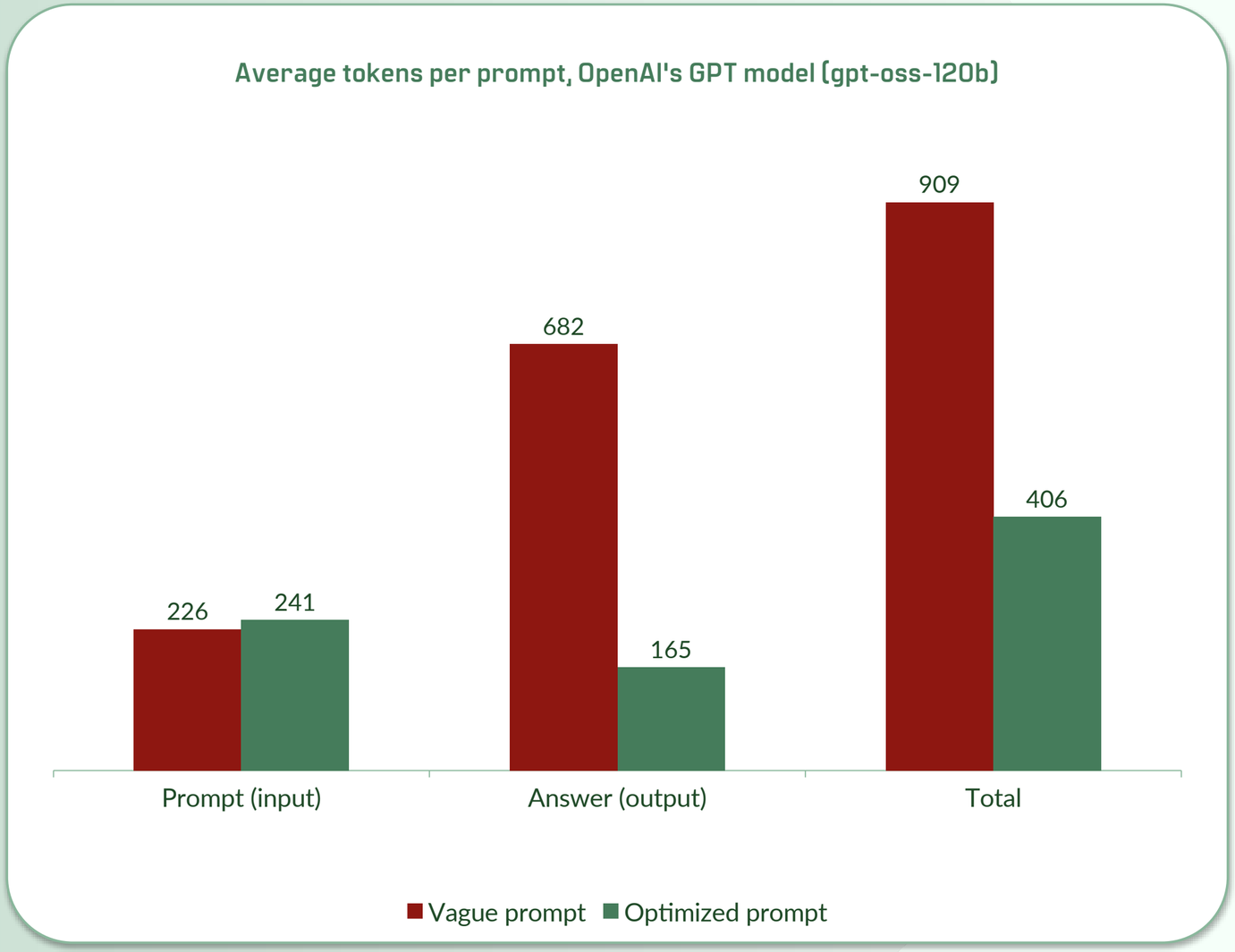
Answer

a page
vague prompt

"good"
optimized

Same AI for both: OpenAI's GPT model (gpt-oss-120b)

DOES IT WORK ON 482 NEW PROMPTS?



39% fewer

total tokens per prompt (482 test prompts)

Answer format stated

3 in 100	95 in 100
vague prompt	optimized

Quality, blind judge (0-10)

8.2	9.0
vague prompt	optimized

Task done right: 67% vs 85%

TESTING

UNIT

798

automatic tests, all pass

- every rule tested on its own
- 4 sample cases per module in the report

INTEGRATION

42 / 50

real-life questions right (vague: 37)

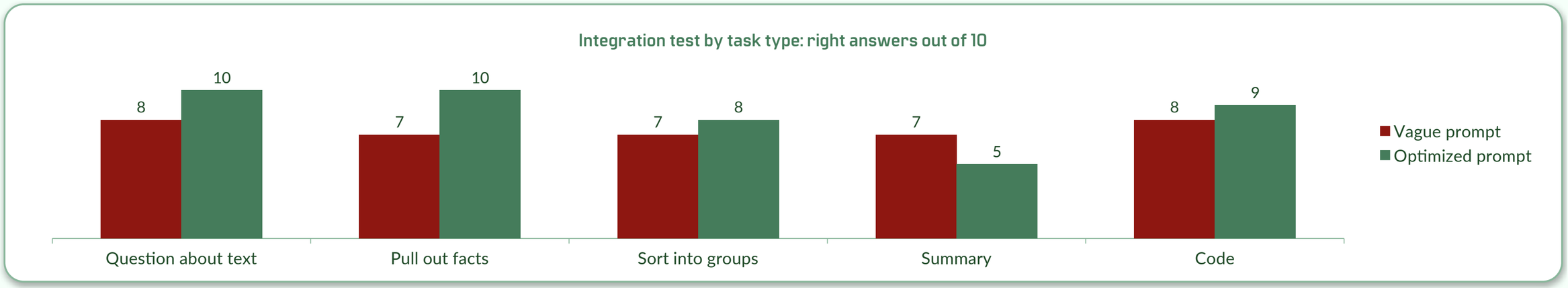
- 10 per task type
- 29.6% fewer tokens

SYSTEM

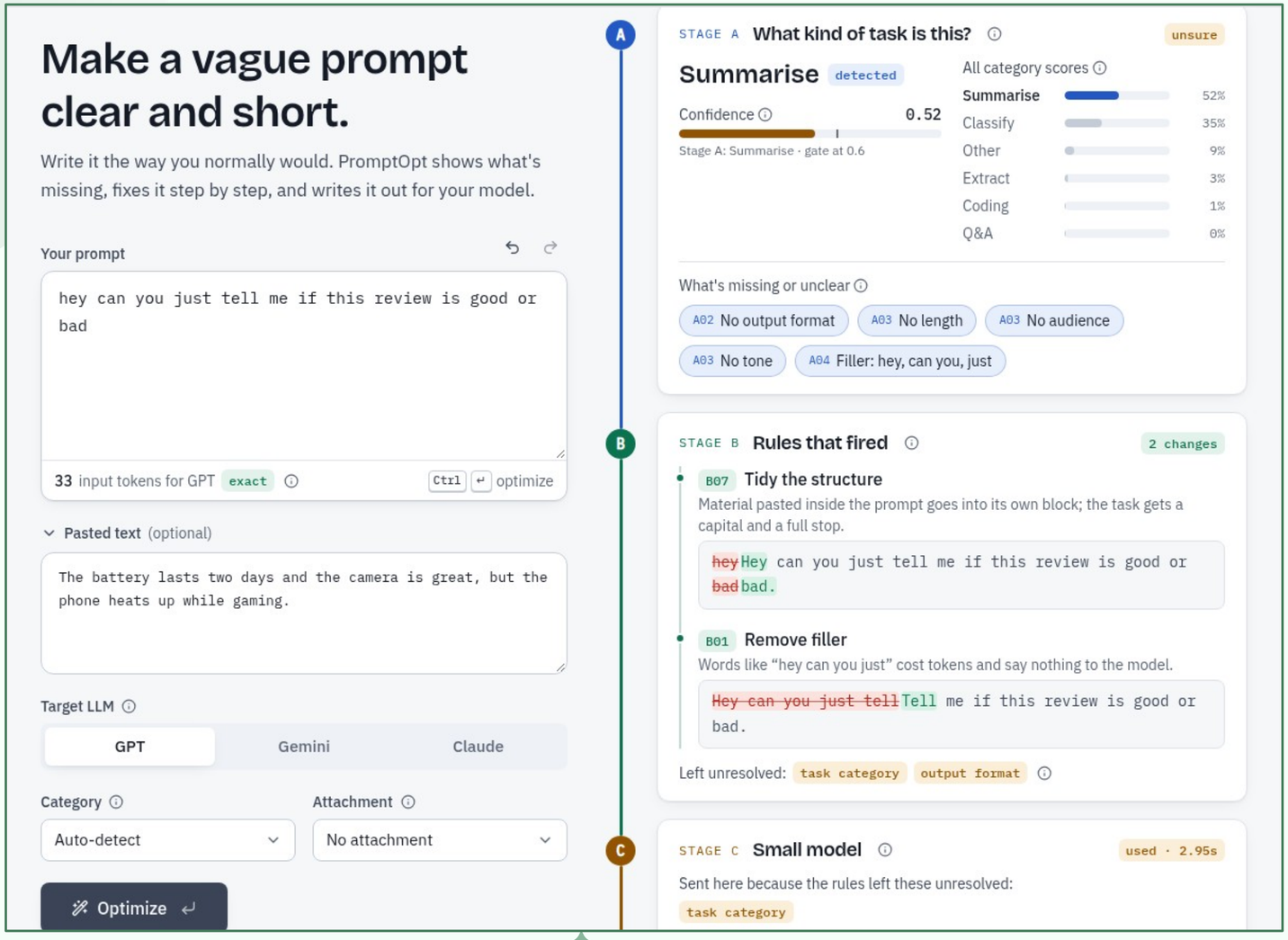
13 / 13

browser tests of the whole app pass

- 482 test prompts run once
- frozen code checked byte by byte



THE WORKING APP



LIVE DEMO

1. **Optimize**
   our review example: every step on screen
2. **Compare**
   both prompts on a real AI
3. **History**
   past prompts, deleted after 30 days
4. **Test suite**
   the 50 real-life questions

CHALLENGES AND NEXT STEPS

WHAT WAS HARD, AND WHAT WE DID

AI-written data dropped facts

computer checks, repair, and we read 330 pairs

The small model made things up

training (5 to 98 in 100) and a check before use

Free AI limits per day

a usage log, pacing, runs that resume next day

4 GB laptop GPU

memory-saving training: 51 minutes

NEXT STEPS

1. **Lean mode**

   add less when the answer is already short

2. **Real GPT, Claude, Gemini**

   with paid API keys

3. **Bigger tests**

   more real-life questions, written by outsiders

4. **Research paper**

   on the method and results

THANK YOU

TECH STACK

Layers	Technologies		
Frontend	React	Tailwind CSS	
Backend & Database	Fast API	PostgreSQL	
LLM & Finetuning	Hugging Face Transformer	PEFT(LoRA)	
NLP & Feature Detection	spaCy	Regex	
Tokenization	Hugging Face Tokenizer	Tiktonken	
Dataset	Dolly	CodeAlpaca	Alpaca

LITERATURE REVIEW

A Systematic Survey of Automatic Prompt OptimizationTechniques

Surveys and defines Automatic Prompt Optimization as a field, offering a taxonomy of existing approaches, but proposes no method of its own or lightweight, self-hosted solution.

~Ramnath et al. (2025)

Towards LLMs Robustness to Changes in Prompt Format Styles

Shows that prompt formatting alone significantly affects LLM output quality, but focuses on training models to be robust, not on optimizing the prompt itself.

~Ngweta et al. (2025)

CAPO: Cost-Aware Prompt Optimization

Uses evolutionary search to balance prompt quality against length and cost, but the approach is computationally expensive, unsuitable for lightweight or self-hosted use.

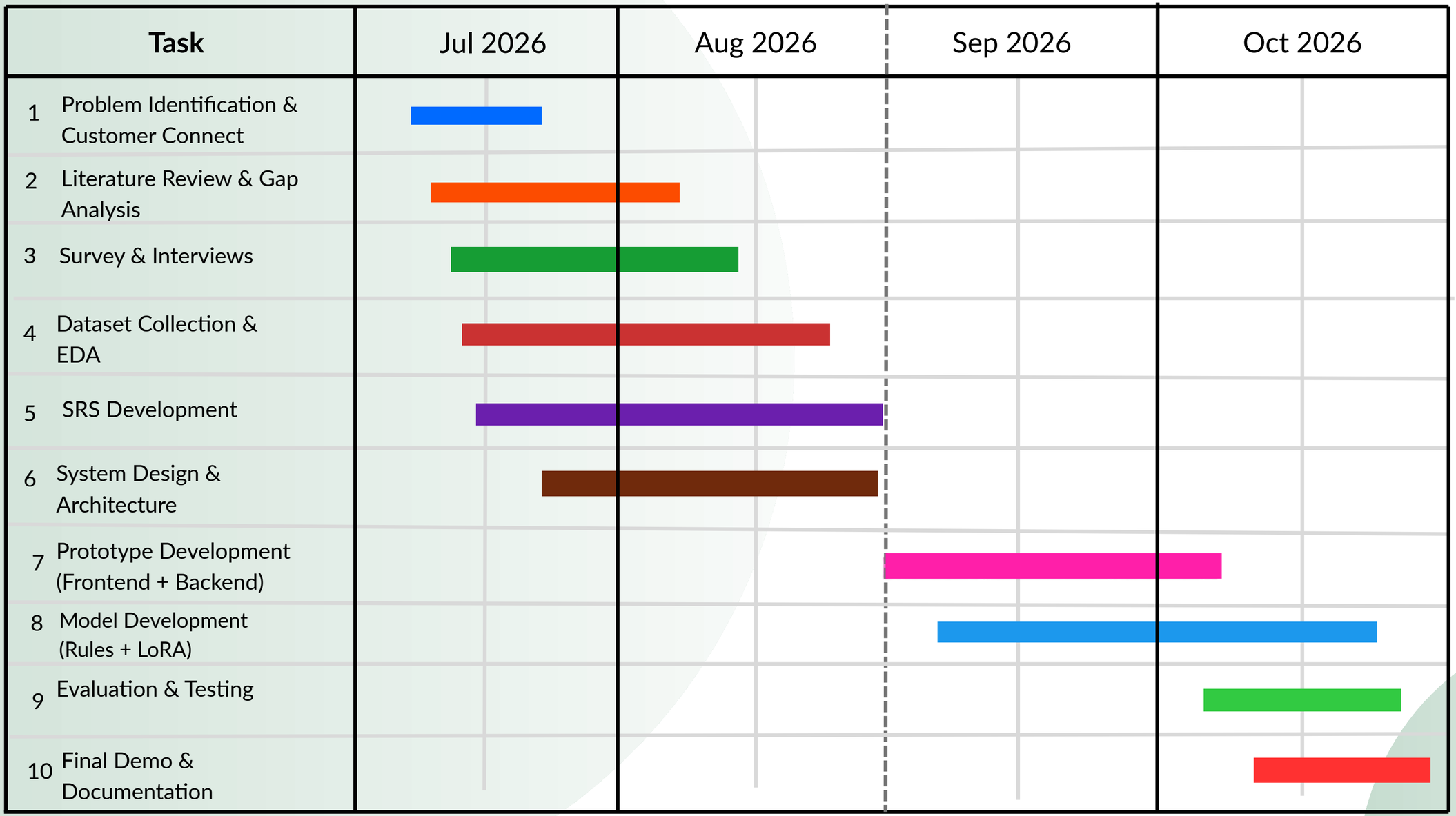
~Zehle et al. (2025)

SDG ALIGNMENT

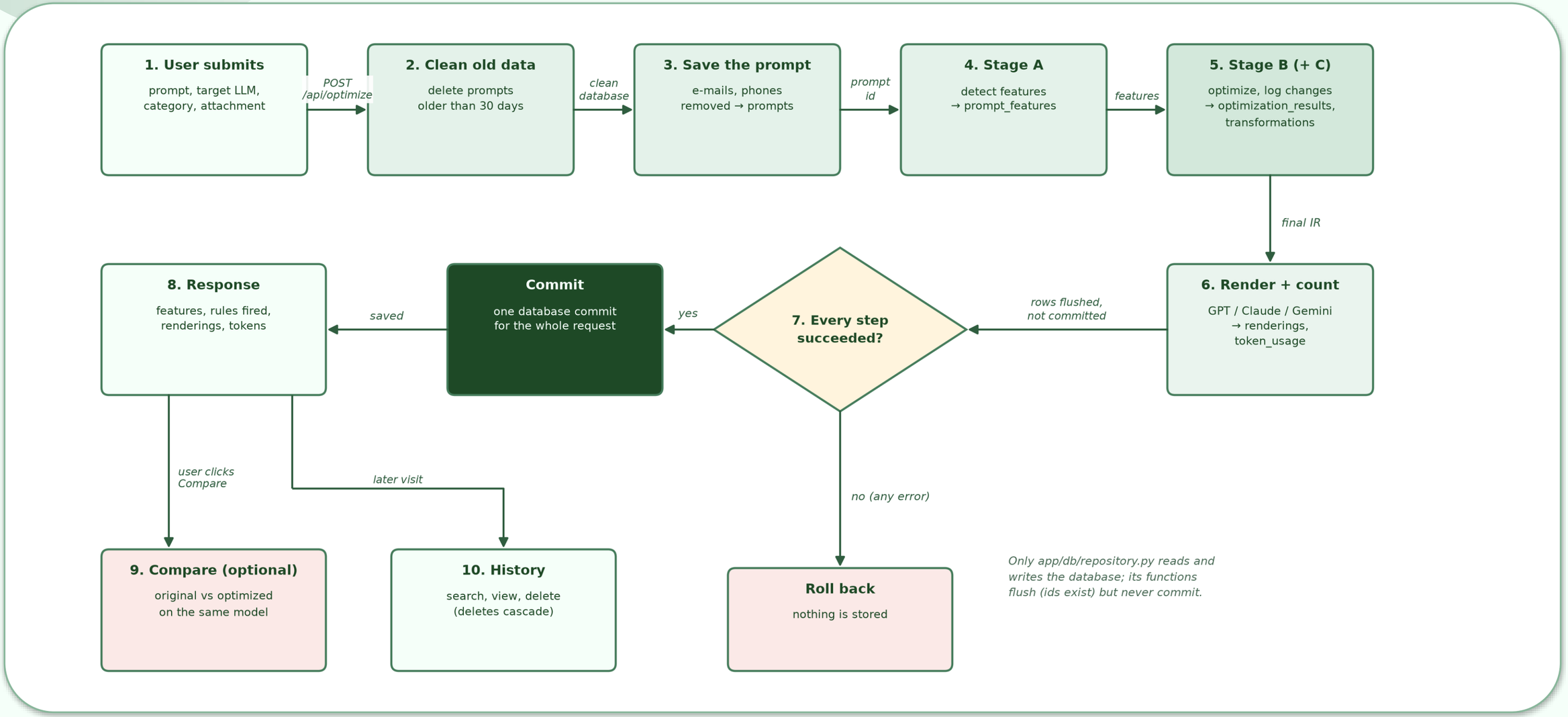


PROMPTOPT – MINI PROJECT-I GANTT CHART

← Work Done Future Work →



APPENDIX: PIPELINE FLOW OF ONE REQUEST



REFERENCES

Ramnath, K., Zhou, K., Guan, S., et al. (2025). A Systematic Survey of Automatic Prompt Optimization Techniques. Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics. https://aclanthology.org/2025.emnlp-main.1681/

Ngweta, L., Kate, K., Tsay, J., & Rizk, Y. (2025). Towards LLMs Robustness to Changes in Prompt Format Styles. Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics: Student Research Workshop (NAACL-SRW). https://aclanthology.org/2025.naacl-srw.51/

Zehle, T., Schlager, M., Heiß, T., & Feurer, M. (2025). CAPO: Cost-Aware Prompt Optimization. Proceedings of the Fourth International Conference on Automated Machine Learning (AutoML), PMLR 293. https://proceedings.mlr.press/v293/zehle25a.html

Taori, R., et al. (2023). Alpaca: A Strong, Replicable Instruction-Following Model. Stanford CRFM. https://github.com/tatsu-lab/stanford_alpaca

Databricks (2023). Databricks Dolly 15k Dataset. https://huggingface.co/datasets/databricks/databricks-dolly-15k

HuggingFace PEFT Documentation — Parameter-Efficient Fine-Tuning (LoRA). https://huggingface.co/docs/peft